

Governed AI-Assisted Engineering: Graduated Human Oversight for Agentic Code Generation in Regulated Domains

Dr. Richard Kang 
DoiT International
richard@doit.com

Abstract—The adoption of agentic AI coding systems — where autonomous agents generate, review, test, and deploy code with minimal human intervention — creates a governance challenge in regulated industries. Existing frameworks address AI-assisted development maturity or the productivity-reliability tension but offer no mechanism for calibrating human oversight intensity to regulatory impact. We present the Governed AI-Assisted Engineering (GAIE) framework, a three-tier graduated human oversight model for agentic code generation in regulated domains. GAIE introduces the Oversight Classification Model (OCM), a deterministic decision function that classifies code generation tasks by regulatory impact, customer proximity, reversibility, and data sensitivity to route them through one of three oversight tiers: human-in-the-loop (strategic functions), human-over-the-loop (customer-impacting), or automated-with-monitoring (internal). Each tier defines required evidence artifacts for compliance auditability. We map GAIE against the Bank of Thailand’s 2025 AI risk-management policy and demonstrate cross-jurisdiction applicability to MAS (Singapore), NIST AI RMF, ISO/IEC 42001, and the EU AI Act. Evaluation through regulatory coverage analysis, comparative framework analysis, and analytical productivity modeling suggests that graduated oversight preserves 84–97% of agentic coding velocity (central estimate: 91%) while maintaining compliance evidence coverage for regulated functions. GAIE contributes a framework that explicitly bridges AI-assisted development maturity with regulatory governance through proportionate human oversight.

Index Terms—agentic coding, human-in-the-loop, software engineering governance, AI development lifecycle, regulated industries, graduated autonomy

The views expressed in this paper are those of the authors in their individual capacity as researchers and do not represent the official position, policy, or interpretation of their respective employers or any regulatory authority. This work constitutes academic thought leadership and does not constitute compliance advice or an institutional compliance assessment.

I. INTRODUCTION

A. The Agentic Coding Paradigm Shift

AI-assisted software development has undergone a qualitative transformation. Where early tools such as GitHub Copilot operated at the granularity of a single line or function — suggesting completions within human-directed workflows — modern agentic systems perform multi-step reasoning, autonomous planning, tool use, and long-horizon code generation with minimal human intervention [1], [2]. Systems such as Claude Code, Devin, SWE-agent, OpenHands, and MetaGPT

can receive a high-level task description and autonomously decompose it into subtasks, generate code across multiple files, write tests, debug failures, and prepare deployment artifacts [2], [3].

The speed is unprecedented. Agentic coding systems produce in minutes what human developers produce in hours [4]. Controlled studies report productivity gains of 20–56% on well-scoped tasks [5], though the most rigorous randomized controlled trial documents a 19% slowdown for experienced developers and telemetry across large developer populations shows substantially more pull requests but far longer review times with flat delivery metrics [5]. This tension between generation speed and reliability assurance forms the core challenge this paper addresses.

B. The Governance Challenge in Regulated Domains

Regulated industries — banking, insurance, healthcare, energy — face supervisory expectations that extend to any AI system influencing decisions with customer or public impact. Financial regulators including the Bank of Thailand (BOT), the Monetary Authority of Singapore (MAS), the UK’s Prudential Regulation Authority (PRA), and the US Office of the Comptroller of the Currency (OCC) require human oversight for high-impact AI-informed decisions [6], [7], [8].

When AI agents write code that *governs* regulated functions — credit approval logic, customer onboarding workflows, anti-money laundering screening rules — the oversight question transfers from model output governance to development pipeline governance. The institution remains responsible for code running in production regardless of whether a human or an agent wrote it [9].

The Bank of Thailand’s 2025 AI risk-management policy is explicit: human participation or human oversight is required whenever AI is used for strategic functions, defined to include credit approval, account opening approval, and approval of deposits, withdrawals, or transfers [6]. The policy further mandates lifecycle-wide controls across data quality, model evaluation, explainability, and AI-specific cyber defenses.

C. Scope Boundary

Before proceeding, we establish what GAIE governs and what it does not:

GAIE governs the software engineering process in which AI agents generate code. Specifically, it addresses: the classification of code generation tasks by their regulatory risk profile; the level of human oversight applied during the generation-to-deployment lifecycle; and the evidence artifacts produced at each stage for regulatory auditability.

GAIE does not replace:

- **Model risk management (MRM)** for production AI/ML models that make customer-facing decisions — these remain subject to OCC SR 11-7 / MAS model validation requirements
- **Cybersecurity review** — GAIE’s security scans are generation-time controls, not a substitute for enterprise security architecture review
- **SDLC controls** — GAIE layers governance *on top of* existing software development lifecycle controls; it does not replace them
- **Regulatory approval of customer-facing AI systems** — governance of customer-facing AI falls under separate regulation (e.g., BOT §4.3 Principle 5), not GAIE

This distinction is important: GAIE governs the *development pipeline* (how code is written and deployed), not the *production behavior* of AI systems that code may implement.

D. The Problem: Uniform Oversight Does Not Scale

Two naive approaches to governing agentic code generation both fail:

“**Review everything**” creates velocity collapse. Atlasian’s deployment of human-in-the-loop software development agents found that high cost of human feedback and integration challenges significantly constrained the value proposition [10], [11]. When every AI-generated artifact requires human review at traditional code-review speed, the productivity benefit of agentic coding is entirely consumed by review overhead — creating what Farrag terms the “Productivity-Reliability Paradox” [5].

“**Review nothing**” creates regulatory non-compliance and unauditable code. Without governance, AI-generated code affecting strategic functions reaches production without evidence that human oversight occurred — a direct violation of regulatory expectations [6], [7].

The analogy to automotive autonomy is instructive. SAE J3016 [12] defines six levels of driving automation, each with different human attention requirements. The automotive industry does not debate “should cars be autonomous or not?” — it asks “what level of autonomy is appropriate for this driving context?” We apply the same principle to code generation: not “should agents code autonomously?” but “what level of oversight is appropriate for this code’s regulatory context?”

E. Contributions

This paper makes the following contributions:

- 1) **The Governed AI-Assisted Engineering (GAIE) framework** — a three-tier graduated human oversight model for agentic code generation that preserves velocity

for low-risk tasks while enforcing human accountability for regulated functions.

- 2) **The Oversight Classification Model (OCM)** — a formal, deterministic decision function that routes code generation tasks to appropriate oversight tiers based on four risk dimensions. We establish key properties (monotonicity, fail-safety under correct or uncertain metadata, completeness) by construction.
- 3) **A per-tier evidence artifact model** — specifying what compliance evidence each tier produces, enabling regulatory auditability as a byproduct of development rather than a separate workstream.
- 4) **Regulatory mapping and cross-jurisdiction analysis** — providing a traceability mapping from GAIE components to selected regulatory control expectations across five frameworks.
- 5) **Multi-method evaluation** — including regulatory coverage analysis, comparative framework analysis, and analytical productivity impact estimation with sensitivity analysis.

F. Running Example

Throughout this paper, we use a fictional running example drawn from a generic regulated bank implementing agentic coding. This example is illustrative and does not represent any specific institution.

Scenario (illustrative — not based on any specific institution): A composite regulated commercial bank adopts an agentic coding system for its engineering teams. The bank’s codebase includes: (a) credit decisioning microservices that determine loan approvals, (b) customer-facing mobile banking APIs, (c) internal developer tooling and CI/CD pipelines. The bank must comply with national AI risk management regulations that require human oversight for strategic functions.

G. Paper Organization

Section 2 reviews background and related work. Section 3 presents the GAIE framework including the OCM, three-tier oversight model, and evidence artifact model. Section 4 maps GAIE to regulatory requirements across jurisdictions. Section 5 describes a reference implementation architecture. Section 6 presents evaluation results. Section 7 discusses implications, limitations, and future work. Section 8 concludes.

II. BACKGROUND AND RELATED WORK

A. Evolution of AI-Assisted Software Development

We identify four phases in the evolution of AI-assisted software development, depicted in Figure 1.

Phase 1 — AI-assisted autocomplete (2021–2023). Tools such as GitHub Copilot [13] and TabNine suggest single-line or function-level completions within human-directed workflows. Governance is implicit — every suggestion requires explicit human acceptance. Productivity studies report 55% faster task completion for basic coding tasks [14].

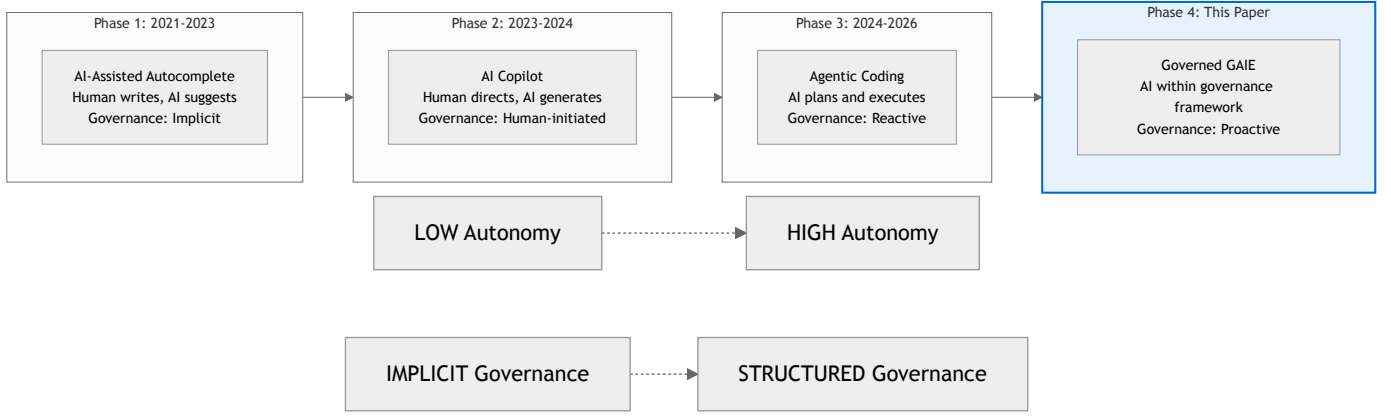


Fig. 1. Evolution of AI-Assisted Software Development

Phase 2 — AI copilot (2023–2024). Conversational AI assistants integrated into IDEs generate multi-line code blocks in response to natural-language prompts. Governance remains human-initiated [15].

Phase 3 — Agentic coding (2024–2026). Autonomous systems decompose high-level tasks, make multi-step decisions, use tools, and produce complete implementations [1], [2], [3]. SWE-bench evaluations show these systems resolving 20–70% of real GitHub issues autonomously [16].

Phase 4 — Governed AI-assisted engineering (this paper). Agentic systems operate within a governance framework that applies proportionate oversight calibrated to the regulatory impact of each task.

B. Human-in-the-Loop in Software Engineering

CentaurEval [4] introduces a benchmark for measuring the value of human presence in agentic coding, finding that human reasoning combined with AI efficiency outperforms either alone on complex tasks. Takerngsaksiri et al. [10] and Pasukmit et al. [11] report on Atlassian’s deployment of HITL agents, identifying high cost and the need for structured interaction protocols. Mozannar et al. [17] present Magentic-UI with co-planning and action-guarding patterns. Together these works establish that human presence adds value but is costly when applied uniformly — motivating oversight that is calibrated rather than constant.

C. Maturity Models for AI in Software Engineering

Anderson [18] presents the AI Codebase Maturity Model (ACMM), a six-level framework (Level 0–5) describing how codebases evolve from basic AI-assisted coding to fully autonomous systems. ACMM describes *what level of AI assistance exists* but not *how to govern it proportionately*. Zohaib et al. [19] present a layered maturity model for agentic AI in 6G software businesses. The CMMI lineage [20] provides precedent for staged maturity models.

D. The Productivity-Reliability Tension

Farrag [5] identifies the “Productivity-Reliability Paradox”: AI coding shows 20–56% productivity gains on well-scoped

tasks but 19% slowdown for experienced developers, with 91% longer review times. Farrag proposes specification-driven governance as the resolution — applying uniformly regardless of regulatory risk. Our work observes that the paradox is an artifact of *uniform* governance applied to *heterogeneous* risk.

E. Graduated Autonomy in Safety-Critical Domains

The principle of graduated autonomy has precedent: SAE J3016 [12] (automotive, L0–L5), Parasuraman et al. [21] (aviation), Zabolotnii et al. [22] (clinical AI staged autonomy), and NUREG [23] (nuclear safety classification). These domains share: **oversight intensity should be proportionate to consequence severity**. GAIE applies this principle to software engineering.

F. Governance Frameworks for Agentic AI in Software Engineering

Several recent works address fragments: Ait et al. [24] (governance DSL), Treude et al. [9] (accountability gaps), Hasanli [25] (TDD governance), Boyuan et al. [26] (dual-helix governance), Zietsman et al. [27] (governance prompt quality), and Alenezi et al. [28] (SE reorientation). None provides a comprehensive tiered oversight framework calibrated to regulatory requirements.

G. Regulatory Frameworks for AI

The Bank of Thailand [6] requires human participation for strategic functions, lifecycle governance, and FEAT principles. MAS FEAT [7], NIST AI RMF [29], ISO/IEC 42001 [30], BIS [31], the FSB [32], and the EU AI Act [33] all mandate *proportionate* oversight but do not specify *how* to implement proportionality in development workflows. GAIE fills this gap.

H. Summary and Gap Statement

No existing work combines: (a) graduated human oversight, (b) calibrated to regulatory impact classification, (c) for agentic code generation specifically, (d) with per-tier evidence artifacts, (e) validated against real regulatory frameworks. Table I summarizes.

TABLE I
RELATED WORK COMPARISON

Dimension	ACMM	Farrag	Atlassian	Zabolotnii	SAE	GAIE
Graduated oversight	—	—	Uniform	Clinical	Driving	By reg. impact Financial, cross- juris. Per-tier OCM: 4 dim. 3 theorems
Regulatory compliance	—	—	—	Medical	Auto safety	
Evidence artifacts	—	Specs	—	Evidence	—	
Classification model	Feedback	—	—	Risk	Context	
Formal properties	—	—	—	—	—	

III. THE GOVERNED AI-ASSISTED ENGINEERING (GAIE) FRAMEWORK

A. Framework Overview

Figure 2 presents the GAIE framework architecture. The framework operates as a governance layer between task intake and code deployment, routing each code generation task through the appropriate oversight tier based on its risk classification.

B. Design Principles

GAIE is built on five design principles:

- P1. Proportionality.** Oversight intensity scales with regulatory impact.
- P2. Evidence-by-design.** Every oversight event produces auditable artifacts automatically.
- P3. Fail-safe default.** When classification is uncertain, the OCM escalates to a higher oversight tier, never a lower one.
- P4. Separation of generation and approval.** The agent that generates code cannot approve its own deployment.
- P5. Regulatory defensibility.** Every framework element maps to at least one specific supervisory requirement.

C. The Oversight Classification Model (OCM)

1) *Formal Definition:* **Definition 1 (OCM).** The Oversight Classification Model is a total function:

$$\text{OCM} : T \rightarrow \{\text{Tier1}, \text{Tier2}, \text{Tier3}\} \quad (1)$$

where T is the set of all code generation tasks.

Definition 2 (Risk Feature Space). Each task $t \in T$ is characterized by a risk feature vector:

$$\varphi(t) = (\text{RI}(t), \text{CP}(t), \text{RV}(t), \text{DS}(t)) \quad (2)$$

2) *Classification Decision Function:* Figure 3 presents the OCM classification decision tree.

3) *Formal Properties:* **Theorem 1 (Monotonicity).** For any task t , increasing any risk dimension never decreases the assigned oversight tier.

Proof sketch. The classification rules are ordered by decreasing tier assignment. If $\varphi(t_1) \leq \varphi(t_2)$ component-wise, then $\text{OCM}(t_1) \leq \text{OCM}(t_2)$. $\square$

Theorem 2 (Fail-Safety under correct or uncertain metadata). If risk dimensions are correctly observed, or if uncertainty is detected below the confidence threshold, OCM does not under-classify tasks relative to their true risk profile.

Algorithm 1: OCM Classification

Input: Task t , Registry R , Threshold θ
Output: Tier assignment

```

 $ri \leftarrow \text{regulatory\_impact}(t, R);$ 
 $cp \leftarrow \text{customer\_proximity}(t, \text{graph});$ 
 $rv \leftarrow \text{reversibility}(t);$ 
 $ds \leftarrow \text{data\_sensitivity}(t);$ 
 $c \leftarrow \text{confidence}(ri, cp, rv, ds);$ 
if  $ri = \text{STRATEGIC}$  then
  | return Tier1
end
if  $cp = \text{DIRECT}$  then
  | return Tier2
end
if  $cp = \text{INDIRECT} \wedge ds = \text{PERSONAL}$  then
  | return Tier2
end
if  $cp = \text{INDIRECT} \wedge rv = \text{IRREVERSIBLE}$  then
  | return Tier2
end
if  $c < \theta$  then
  | return Tier1 ; // fail-safe
end
if  $\exists \text{ path in dependency\_graph}(t) \text{ to strategic}$  then
  | if  $\text{path\_length} = 1$  then
  | | return Tier1
  | end
  | else
  | | return Tier2
  | end
end
return Tier3;
```

Proof sketch. Two cases: (a) Correct metadata: priority-ordered rules assign the tier corresponding to the highest-risk dimension present. (b) Uncertain metadata: $c < \theta \Rightarrow$ Tier1. *Residual risk:* incorrect but confident metadata can cause misclassification. $\square$

Theorem 3 (Totality). Every task $t \in T$ receives exactly one tier assignment.

Proof sketch. The decision function terminates unconditionally with RETURN Tier3. The function is deterministic. $\square$

4) Running Example Application (Illustrative):

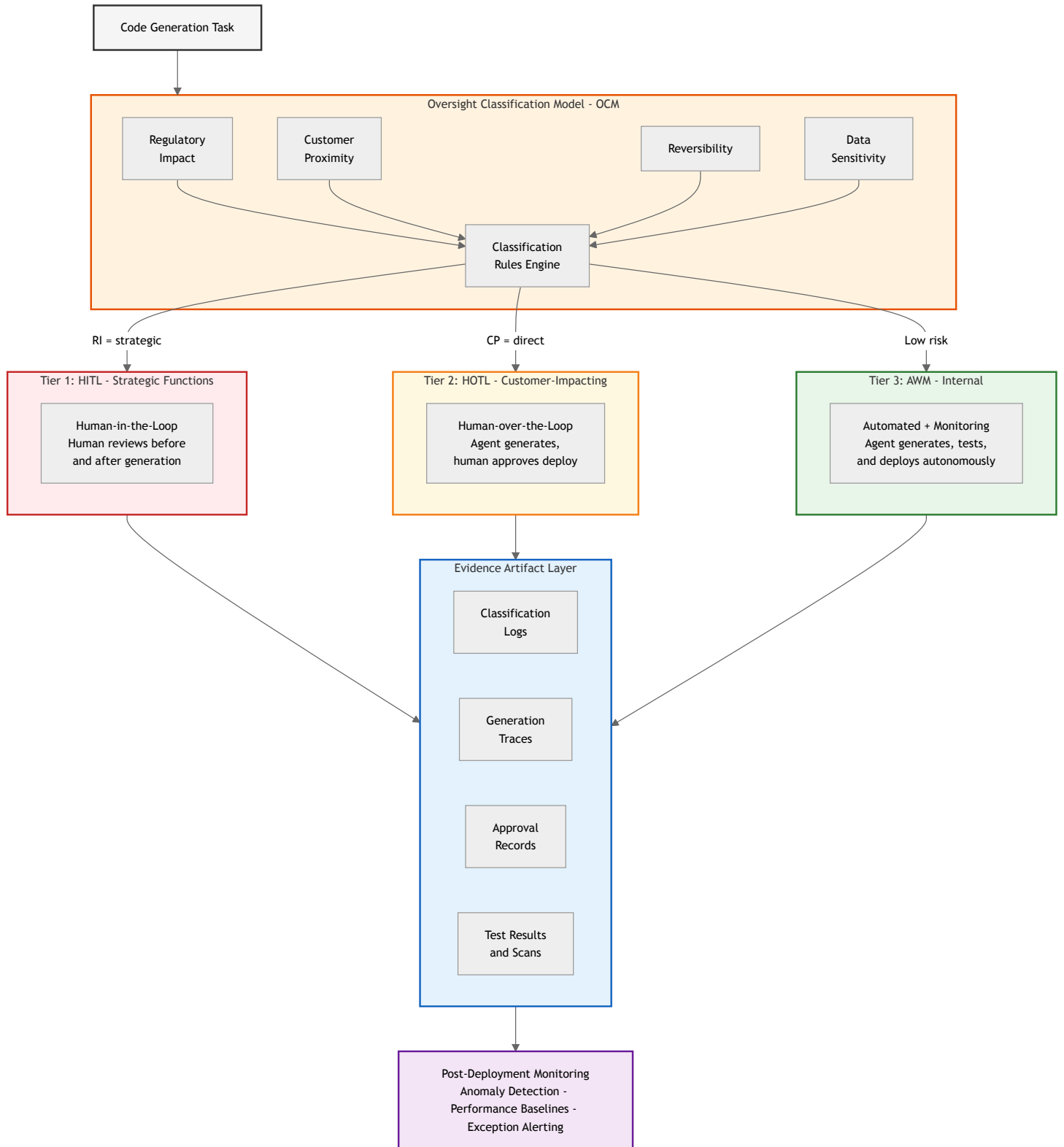


Fig. 2. GAIE Framework Architecture Overview

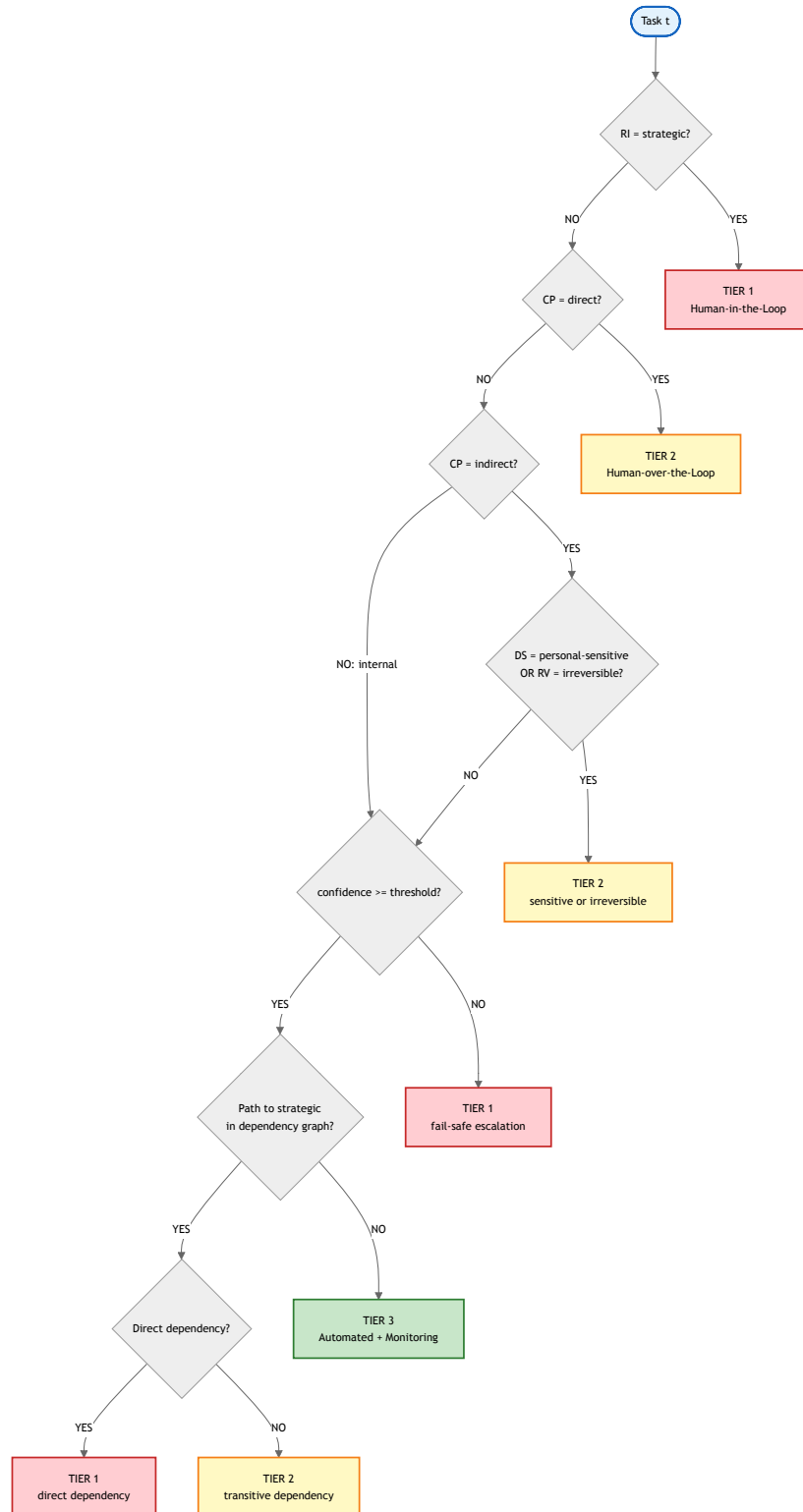


Fig. 3. OCM Decision Tree

TABLE II
OCM RISK DIMENSIONS

Dimension	Domain	Definition
Regulatory Impact (RI)	{strategic, non-strategic}	Whether the code affects functions a regulator classifies as requiring human oversight
Customer Proximity (CP)	{direct, indirect, internal}	Whether the code directly affects customer-facing systems, indirectly supports them, or is purely internal
Reversibility (RV)	{irreversible, partial, full}	Whether deployed code can be rolled back without customer harm
Data Sensitivity (DS)	{personal, business, public}	Whether code generation context includes regulated personal data

TABLE III
OCM CLASSIFICATION — ILLUSTRATIVE BANK SCENARIO

Task	RI	CP	RV	DS	Tier
Modify credit approval threshold	strategic	direct	irreversible	personal	1
Update mobile banking transfer UI	non-strat.	direct	full	business	2
Add field to internal logging	non-strat.	internal	full	public	3
Refactor AML screening rule	strategic	indirect	partial	personal	1
New CI/CD pipeline for test env	non-strat.	internal	full	public	3
API rate-limiting for customers	non-strat.	direct	full	business	2

D. Three-Tier Graduated Oversight

Figure 4 illustrates the human involvement, agent behavior, and evidence flow at each tier.

1) *Tier 1: Human-in-the-Loop (HITL)*: **Trigger**: RI = strategic. The coding agent halts and escalates control (RE-TURN_CONTROL). The human reviews the proposed approach, modifies if needed, and explicitly approves before generation proceeds. At deployment, the human again reviews all evidence and authorizes.

Evidence produced: Classification log, RE-TURN_CONTROL event, human identity, review timestamps, approval decision with rationale, modification diff, deployment authorization with cryptographic signature.

2) *Tier 2: Human-over-the-Loop (HOTL)*: **Trigger**: CP = direct, OR (CP = indirect AND DS = personal), OR (CP = indirect AND RV = irreversible). The agent generates and tests autonomously; a human approves deployment.

Evidence produced: Classification log, generation trace, test results, security scans, human deployment approval.

3) *Tier 3: Automated with Monitoring (AWM)*: **Trigger**: All risk dimensions indicate low impact. Full autonomous pipeline with post-deployment monitoring and exception-based human escalation.

Evidence produced: Pipeline log, test results, deployment record, monitoring baseline, anomaly absence confirmation.

E. Evidence Artifact Model

Figure 5 shows the cryptographically-linked evidence chain.

F. Framework Evolution: Tier Reclassification

Figure 6 depicts the tier reclassification lifecycle. Downward reclassification requires $N \geq 20$ consecutive clean deployments, zero anomalies, rejection rate below 5%, and second-line compliance approval. Upward reclassification triggers immediately on monitoring anomaly, regulatory change, scope expansion, security incident, or new strategic dependency.

TABLE IV
PER-TIER EVIDENCE REQUIREMENTS

Artifact	T1	T2	T3	Rationale
OCM classification log	Req.	Req.	Req.	Auditability
Generation trace	Req.	Req.	Req.	Explainability
Human reviewer ID	Req.	Req.	—	Accountability
Security scan results	Req.	Req.	Req.	Cyber controls
Test execution results	Req.	Req.	Req.	Evaluation
Deploy authorization	Signed	Signed	Auto	Approval
Monitoring record	Req.	Req.	Req.	Ongoing risk
Anomaly detection	Req.	Req.	Req.	Continuous mgmt

G. Supporting Control Dimensions

Three cross-cutting controls operate across all tiers:

Data boundary enforcement. Pre-transmission filtering (regex + NER-based PII detection), access scoping, and inference isolation.

Prompt/response safety controls. Aligned with OWASP LLM Top 10 [34] and MITRE ATLAS [35]: prompt injection detection (LLM01), output validation for backdoors and ex-filtration (ATLAS T0040–T0048), sandboxed execution, and supply chain validation (LLM05).

Immutable audit trail. Complete provenance chain from task request through deployment and monitoring.

H. Threat Model for GAIE

Figure 7 presents the threat model. We identify four threat categories:

T1. Classification evasion. Mitigation: dependency analysis, confidence threshold, fail-safe default.

T2. Evidence tampering. Mitigation: append-only storage, cryptographic hash chain.

T3. Governance fatigue. Mitigation: review duration tracking, modification rate monitoring.

T4. Registry staleness. Mitigation: regulatory override protocol, quarterly review.

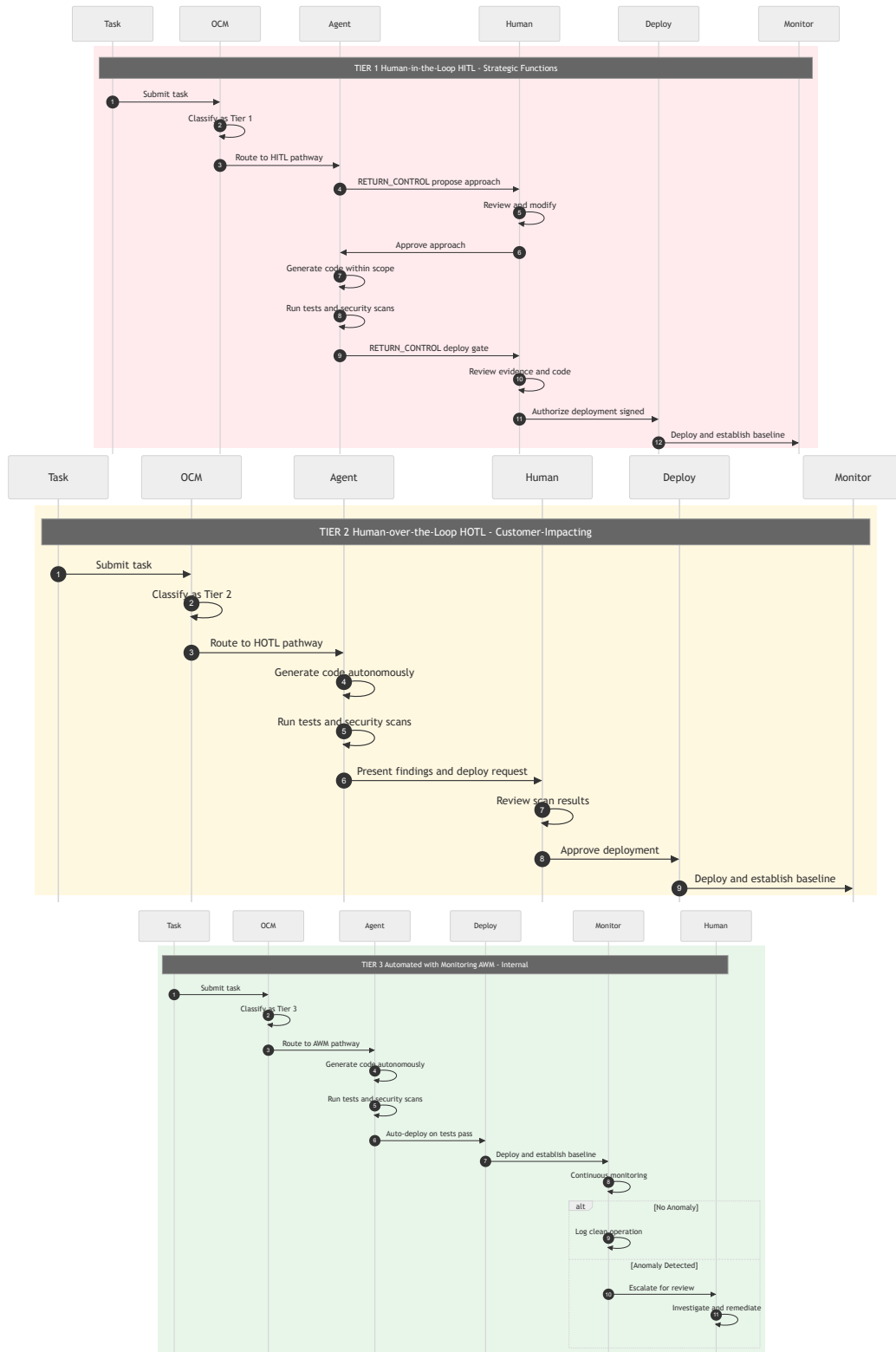


Fig. 4. Three-Tier Oversight Model — Sequence Diagrams

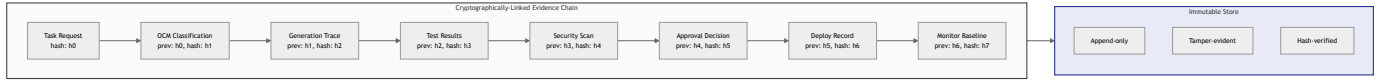


Fig. 5. Evidence Chain Integrity Model

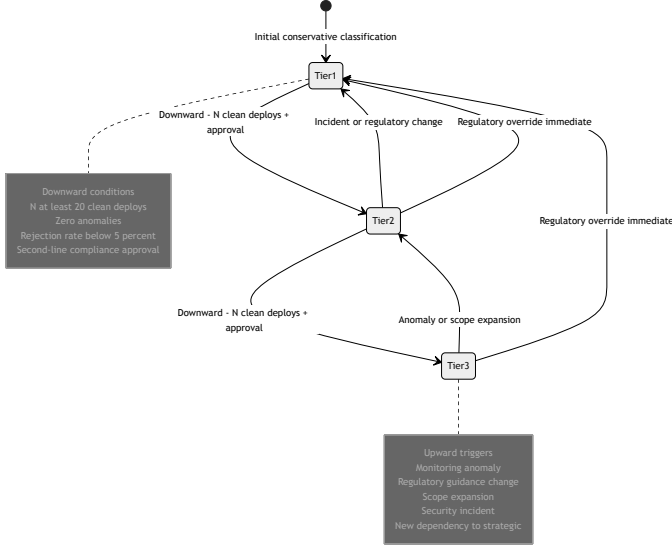


Fig. 6. Tier Reclassification Lifecycle

IV. REGULATORY MAPPING

A. Bank of Thailand AI Risk Management Policy (2025)

We provide a traceability mapping from GAIE components to applicable BOT requirements (Table V).

Traceability assessment: Based on the authors' reading of the publicly available BOT circular, GAIE's design addresses 9 of 10 applicable control domains. This represents academic analysis of published regulatory text and has not been validated by BOT examiners or any regulated institution's compliance function. It should not be construed as a compliance determination.

B. Cross-Jurisdiction Applicability

Figure 8 illustrates the mapping. Table VI details functional correspondence.

Important caveat: These mappings represent the authors' analysis and have not been validated by the respective regulatory authorities.

C. Generalizability Beyond Financial Services

The OCM's four risk dimensions are domain-agnostic. Only the regulatory function registry changes per domain. Examples in banking are drawn from publicly available regulatory guidance and are illustrative — they do not represent any specific institution's classification decisions.

V. REFERENCE IMPLEMENTATION ARCHITECTURE

A. Architecture Overview

Figure 9 presents the reference implementation architecture.

The architecture follows a multi-agent supervisor pattern with five subsystems: supervisor agent, collaborator agents, human interaction layer, evidence capture layer, and monitoring subsystem.

B. Key Implementation Mechanisms

C. Recommended Adoption Sequence

Figure 10 presents the recommended phased adoption. Duration is institution-specific.

Phase 1: Instrument. Deploy evidence capture and monitoring before granting agents deployment permissions.

Phase 2: Classify. Run OCM in shadow mode. Validate against human judgment. Tune θ .

Phase 3: Tier 3 enforcement. Activate for internal tooling.

Phase 4: Tier 2 enforcement. Extend to customer-impacting code with human deployment approval.

Phase 5: Tier 1 enforcement. Extend HITL governance to strategic-function code.

Phase 6: Optimize. Reclassify based on accumulated evidence.

D. End-to-End GAIE Walkthrough

Table VIII traces three illustrative tasks through every GAIE stage. All task descriptions, system names, and parameters are fictional.

VI. EVALUATION

A. Evaluation Methodology

We evaluate GAIE through three complementary methods: (1) regulatory coverage analysis, (2) comparative framework analysis, and (3) analytical productivity impact estimation. This multi-method approach follows established practice for framework contributions [20], [36]. The evaluation is *analytical* — no production deployment data is available.

B. Regulatory Coverage Analysis

Of 18 enumerated BOT control requirements (6 organizational, 12 development/security), GAIE provides functional traceability to all 17 applicable requirements. The one non-applicable requirement (customer disclosure) is correctly excluded per the scope boundary (§1.3).

Important qualification: Traceability does not equal compliance. This demonstrates that GAIE's design *addresses* each requirement category; it does not constitute a compliance certification.

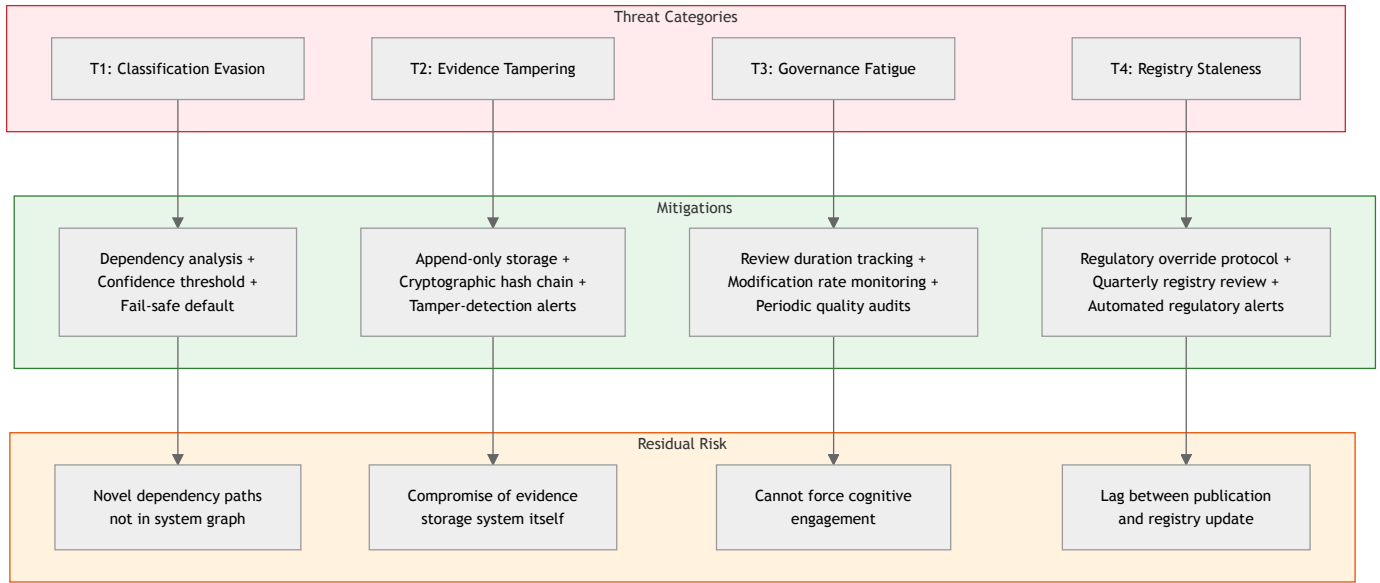


Fig. 7. GAIE Threat Model

TABLE V
BOT POLICY → GAIE TRACEABILITY MAPPING

BOT Requirement	GAIE Component	Evidence
Human participation (§4.3 P4)	OCM → Tier 1 + HITL	Classification + approval
Lifecycle governance (§4.4)	Three-tier model	Per-phase artifacts
FEAT principles (§4.3 P1–4)	OCM + evidence model	Audit trail
Data boundaries (§4.4 Pt.2(1))	Data boundary enforcement	Filtering logs
Testing (§4.4 Pt.2(2))	Gen-validation separation (P4)	Test results
AI cyber controls (§4.4 Pt.2(3))	Prompt/response safety	Attack detection logs
Explainability (§4.3 P2)	Generation trace	Full trace
Three lines of defense	Agent/human/monitor	Role separation

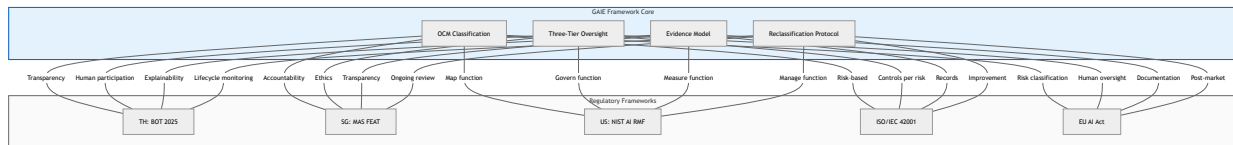


Fig. 8. Cross-Jurisdiction Regulatory Mapping

TABLE VI
CROSS-JURISDICTION TRACEABILITY

Framework	Key Requirement	GAIE Mapping	Gap
MAS FEAT	FEAT principles	OCM transparency + human gates	Dev lifecycle
NIST AI RMF	Govern, Map, Measure, Manage	Four functions mapped	Full traceability
ISO/IEC 42001	Risk-based AI mgmt system	OCM = risk-based approach	Clause 4 separate
EU AI Act	Human oversight (Art. 14)	Tier 1 for high-risk	Conformity not covered
OCC MRM	Model validation	Gen-validation separation	Complements, not replaces

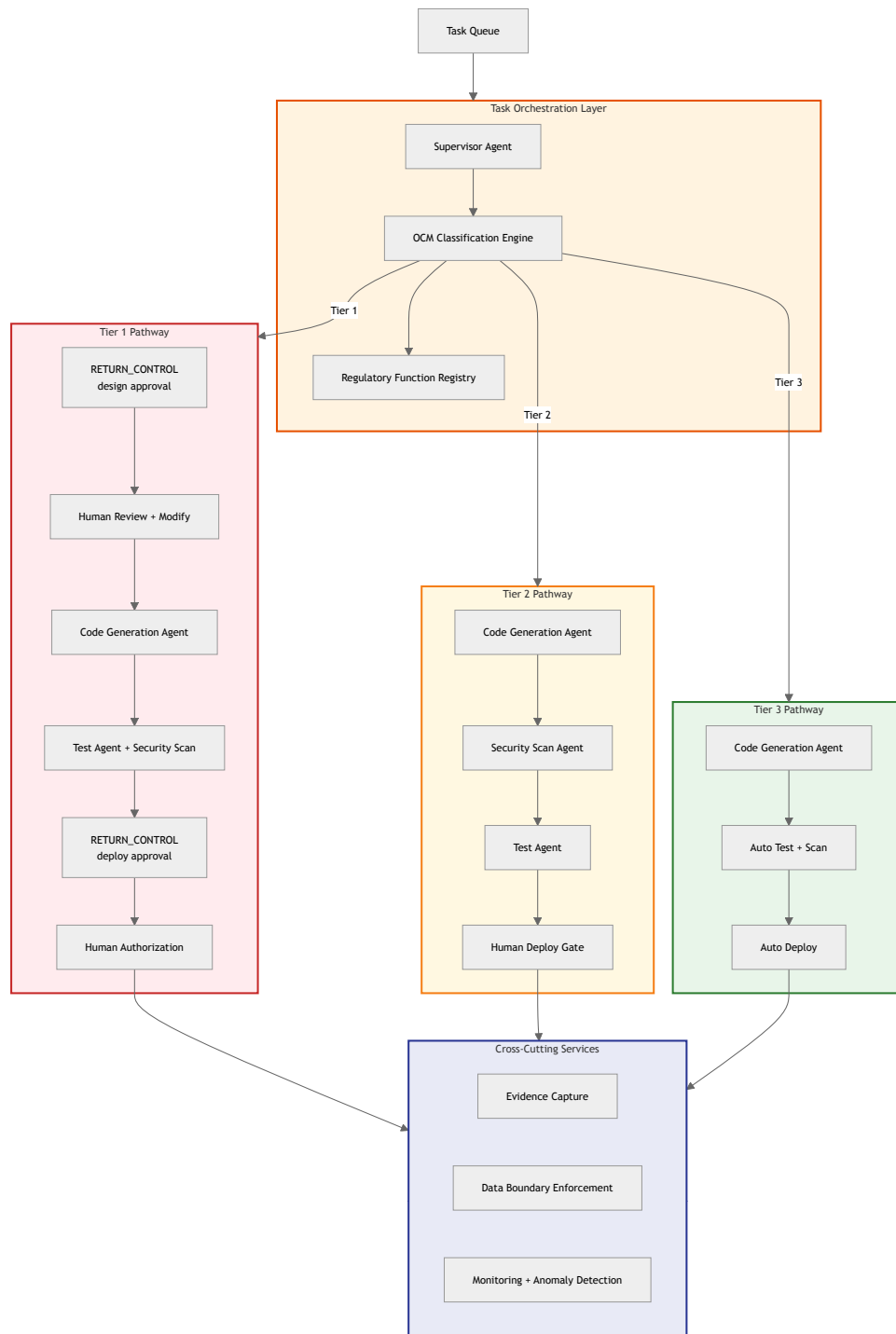


Fig. 9. Reference Implementation Architecture

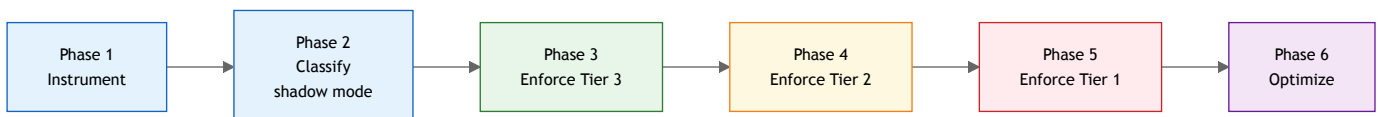


Fig. 10. Phased Adoption Sequence

TABLE VII
GAIE CONCEPTS → IMPLEMENTATION MECHANISMS

Concept	Mechanism	Principle
OCM classification	Rule engine + registry	Deterministic routing
Tier 1 RETURN_CONTROL	Agent escalation API	Halt + notify + await
Tier 2 deploy gate	CI/CD human approval	Block until authorized
Tier 3 auto deploy	Automated quality gates	Deploy on pass
Evidence capture	Invocation logging + events	Cryptographic chaining
Immutable storage	Append-only object store	Write-once, tamper-evident
Data boundaries	Pre-transmission filtering	No sensitive data to external APIs
Gen-validation separation	Distinct agent instances	Author $\neq$ tester

TABLE VIII
END-TO-END GAIE WALKTHROUGH — THREE ILLUSTRATIVE TASKS

Stage	Tier 1: Credit	Tier 2: Mobile	Tier 3: CI/CD
Task	Lower loan threshold	Add date filter	Add retry logic
Risk	RI=strategic, CP=direct	CP=direct, RV=full	CP=internal, RV=full
Result	Tier 1	Tier 2	Tier 3
Generation	Human approves approach first	Autonomous	Autonomous
Human gate	Design + deploy	Deploy only	None
Monitoring	Enhanced 72h	Standard	Basic
Elapsed	~4 hours	~45 min	~8 min

C. Comparative Analysis

Table IX shows feature comparison against related frameworks.

D. Analytical Productivity Impact Estimation

Analytical modeling based on estimated task volume distributions (Table X). These are analytical approximations, not empirical measurements.

Result: Analytical modeling suggests GAIE preserves 84–97% of ungoverned agentic coding velocity (central estimate: ~91%).

E. Consistency with Empirical Baselines

Our estimates are directionally consistent with: Farrag [5] (~81% under uniform governance), Atlassian [10], [11] (high HITL cost), and GitHub Copilot studies [14] (55% faster ungoverned). These remain *analytical approximations*; empirical validation is future work.

VII. DISCUSSION

A. Positioning Against Related Work

GAIE complements ACMM (describes *where*; GAIE prescribes *how*), resolves Farrag’s Paradox (by tiering rather than uniform governance), extends graduated autonomy to SE (from automotive, aviation, clinical domains), and operationalizes regulatory intent (translating principles into engineering processes).

B. The Proportionality Principle

Graduated oversight is superior on four grounds: **regulatory alignment** (aligns with the proportionality principle that regulators have established), **productivity evidence** (HITL adds value on complex judgment tasks, not routine generation [4]),

governance quality (concentrating attention prevents theatrical compliance), and **information theory** (graduated oversight allocates reviewer bandwidth to high-information decisions).

C. Limitations and Threats to Validity

- L1.** Single jurisdiction for primary mapping; cross-jurisdiction lighter.
- L2.** No production deployment data; estimates are analytical.
- L3.** Classification boundary ambiguity at edges.
- L4.** Expert validation pending (5–8 practitioners planned).
- L5.** Technology coupling in reference architecture.
- L6.** Adversarial robustness: confident but incorrect metadata is a failure mode.
- L7.** Regulatory mappings not externally validated by authorities.

D. Implications for Practice

For regulated institutions: Adopt graduated oversight rather than the false dichotomy of “review everything” vs. “ban AI coding.”

For AI coding tool vendors: Build RETURN_CONTROL / escalation mechanisms as first-class features.

For regulatory design (research observation): Our analysis suggests that clarity of regulatory function definitions directly affects governance implementation efficiency. Where strategic function boundaries are clearly defined, institutions can calibrate oversight precisely.

For researchers: Empirical studies needed on OCM accuracy, tier calibration, false-escalation rates, and evidence artifact effectiveness.

E. Future Work

- 1) Empirical validation through production deployment
- 2) Expert practitioner validation (Appendix B instrument)

TABLE IX
COMPARATIVE ANALYSIS RESULTS

Dimension	ACMM	Farrag	Atl.	Zab.	GAIE
Regulatory compliance?	No	Partial	No	Med.	Yes
Productivity for low-risk?	N/A	Spec	No	N/A	Yes
Auditability evidence?	No	Spec	No	Yes	Yes
Proportionate oversight?	No	Uniform	Uniform	Grad.	Yes
Formal properties?	No	No	No	No	Yes
Fail-safe?	No	No	No	Partial	Yes



Fig. 11. Productivity Impact — GAIE vs. Uniform Oversight

TABLE X
PRODUCTIVITY IMPACT MODEL

Parameter	Conserv.	Moderate	Optimistic
Tier 3 volume share	60%	70%	80%
Tier 2 volume share	25%	20%	15%
Tier 1 volume share	15%	10%	5%
Tier 3 velocity	95%	98%	100%
Tier 2 velocity	80%	85%	90%
Tier 1 velocity	45%	55%	65%
Weighted velocity	83.8%	91.1%	96.8%

- 3) OCM automation via static analysis
- 4) Machine-checked proofs (Coq, Lean)
- 5) Extension to healthcare, insurance, energy
- 6) Longitudinal reclassification study
- 7) Integration with GRC platforms
- 8) Multi-model governance

VIII. CONCLUSION

Agentic AI code generation in regulated industries requires governance, but uniform oversight creates a productivity-reliability paradox. GAIE resolves this through graduated human oversight calibrated to regulatory impact.

The OCM routes tasks through three tiers based on regulatory impact, customer proximity, reversibility, and data sensitivity. Human-in-the-loop applies to strategic-function code (estimated 5–15%). Human-over-the-loop applies to customer-impacting code (15–25%). Automated-with-monitoring applies to internal tooling (60–80%). Each tier produces defined evidence artifacts.

Evaluation suggests: (a) functional traceability to all applicable BOT lifecycle requirements, (b) differentiation from existing frameworks, (c) formal OCM properties (monotonicity, fail-safety, totality), and (d) 84–97% velocity preservation (central: 91%) vs. 45–65% under uniform HITL.

GAIE contributes a framework that explicitly bridges AI-assisted development maturity with regulatory governance through proportionate human oversight — making governance the enabler of velocity rather than its constraint.

REFERENCES

- [1] H. Bhati, “Agentic AI in the software development lifecycle: Architecture, empirical evidence, and the reshaping of software engineering,” *arXiv preprint arXiv:2604.26275*, 2026.
- [2] H. Li, “The rise of AI teammates in software engineering (SE) 3.0: How autonomous coding agents are reshaping software engineering,” *arXiv preprint arXiv:2507.15003*, 2025.
- [3] X. Wang *et al.*, “OpenHands: An open platform for AI software developers as generalist agents,” *arXiv preprint arXiv:2407.16741*, 2024.
- [4] H. Luo, “CentaurEval: Benchmarking human-in-the-loop value in agentic coding,” *arXiv preprint arXiv:2512.04111*, 2025.
- [5] S. E. Farrag, “The productivity-reliability paradox: Specification-driven governance for AI-augmented software development,” *arXiv preprint arXiv:2605.01160*, 2026.
- [6] Bank of Thailand, “Risk management in the use of artificial intelligence systems,” Circular ThoPho 3.5994/2568, Sep. 2025.
- [7] Monetary Authority of Singapore, “Principles to promote fairness, ethics, accountability and transparency (FEAT) in the use of AI and data analytics in singapore’s financial sector,” 2018.
- [8] Office of the Comptroller of the Currency, “Supervisory guidance on model risk management,” OCC Bulletin 2011-12, 2011.
- [9] C. Treude, “Accountable agents in software engineering: An analysis of terms of service and a research roadmap,” *arXiv preprint arXiv:2605.04532*, 2026.
- [10] W. Takerngsaksiri, “Human-in-the-loop software development agents,” *arXiv preprint arXiv:2411.12924*, 2024.
- [11] J. Pasukmit, “Human-in-the-loop software development agents: Challenges and future directions,” *arXiv preprint arXiv:2506.11009*, 2025.
- [12] SAE International, “SAE J3016: Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles,” 2021.
- [13] GitHub, “GitHub Copilot: Your AI pair programmer,” 2021.
- [14] S. Peng *et al.*, “The impact of AI on developer productivity: Evidence from GitHub Copilot,” *arXiv preprint arXiv:2302.06590*, 2023.
- [15] OpenAI, “ChatGPT,” 2022.
- [16] C. E. Jimenez *et al.*, “SWE-bench: Can language models resolve real-world GitHub issues?” *arXiv preprint arXiv:2310.06770*, 2023.
- [17] H. Mozannar, “Magentic-UI: Towards human-in-the-loop agentic systems,” *arXiv preprint arXiv:2507.22358*, 2025.
- [18] A. Anderson, “The AI codebase maturity model: From assisted coding to fully autonomous systems,” *arXiv preprint arXiv:2604.09388*, 2026.
- [19] M. Zohaib, “Agentic AI in 6G software businesses: A layered maturity model,” *arXiv preprint arXiv:2508.03393*, 2025.
- [20] CMMI Institute, “CMMI for development, version 1.3,” Software Engineering Institute, 2010.
- [21] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, “A model for types and levels of human interaction with automation,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 30, no. 3, pp. 286–297, 2000.
- [22] S. Zabolotnii, “From black-box confidence to measurable trust in clinical AI: A framework for evidence, supervision, and staged autonomy,” *arXiv preprint arXiv:2604.26671*, 2026.
- [23] U.S. Nuclear Regulatory Commission, “NUREG-0800: Standard review plan for the review of safety analysis reports,” 2020.

- [24] A. Ait, “Towards automated governance: A DSL for human-agent collaboration in software projects,” *arXiv preprint arXiv:2510.14465*, 2025.
- [25] T. Hasanli, “TDD governance for multi-agent code generation via prompt engineering,” *arXiv preprint arXiv:2604.26615*, 2026.
- [26] B. Guan, W. Cui, and L. Juhasz, “A dual-helix governance approach towards reliable agentic artificial intelligence for WebGIS development,” *arXiv preprint arXiv:2603.04390*, 2026.
- [27] C. Zietsman, “Structural quality gaps in practitioner AI governance prompts: An empirical study using a five-principle evaluation framework,” *arXiv preprint arXiv:2604.21090*, 2026.
- [28] M. Alenezi, “Rethinking software engineering for agentic AI systems,” *arXiv preprint arXiv:2604.10599*, 2026.
- [29] National Institute of Standards and Technology, “AI risk management framework (AI RMF 1.0),” NIST AI 100-1, Jan. 2023.
- [30] International Organization for Standardization, “ISO/IEC 42001:2023 — artificial intelligence — management system,” 2023.
- [31] Bank for International Settlements, “Intelligent financial system: How AI is transforming finance,” *BIS Working Paper 1194*, Jun. 2024.
- [32] Financial Stability Board, “The financial stability implications of artificial intelligence,” Nov. 2024.
- [33] European Union, “Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act),” 2024.
- [34] OWASP Foundation, “OWASP top 10 for large language model applications, version 2.0,” 2025.
- [35] MITRE Corporation, “ATLAS: Adversarial threat landscape for AI systems,” 2024.
- [36] B. A. Kitchenham *et al.*, “Preliminary guidelines for empirical research in software engineering,” *IEEE Transactions on Software Engineering*, vol. 28, no. 8, pp. 721–734, 2002.

APPENDIX

Layer 1: Organizational Governance

- A1. Board/senior management accountability
- A2. FEAT principles adoption
- A3. AI usage policy aligned with risk appetite
- A4. Three lines of defense
- A5. Human participation/oversight for strategic functions
- A6. Customer disclosure (not applicable to dev tools)

Layer 2: Development and Security Controls

- B1–B4. Data quality, assessment, leakage prevention, boundaries
- B5–B7. Model performance, testing, ongoing monitoring
- B8–B9. GenAI grounding, explainability
- B10–B12. Prompt filtering, attack testing, emerging threats

GAIE traceability: A1–A5, B1–B12. A6 not applicable (§1.3).

Participant criteria: Employed at regulated FI; role in engineering leadership, compliance, or security architecture; 3+ years experience; subject to tier-1 regulator.

Quantitative (5-point Likert):

- 1) GAIE addresses governance concerns for AI-assisted development
- 2) Adoptable (as-is or adapted) for governing agentic coding
- 3) Three-tier model appropriately calibrated
- 4) Evidence artifacts sufficient for regulatory examinations
- 5) OCM dimensions capture relevant risk factors
- 6) Fail-safe default appropriate for risk tolerance
- 7) Reclassification protocol provides adequate safeguards

Open-ended: Gaps not addressed; tier boundary changes; comparison to current approach; adoption barriers; jurisdiction-specific gaps.